

Investigate potential design patterns that could be useful for your project. Justify their need in your project.

1. Strategy pattern:

This pattern could be useful because “routing” is not one fixed rule in our app. The best path depends on the transportation mode the user selects (walking, car, bus, metro), and even inside walking there can be different routes to take or constraints (for example: avoiding stairs for accessibility, preferring indoor corridors in winter, etc.). If all of that logic is being written as one huge block of if/else statements, the routing code becomes hard to read since it becomes a block of interlinked messy code that can confuse anyone trying to read and make sense of it. Furthermore, it can be risky to change because adding a new mode or a new constraint can break existing modes. With the Strategy design pattern, each mode can have its own routing algorithm in its own class (WalkingRouteStrategy, DrivingRouteStrategy, TransitRouteStrategy, etc.), and the app simply chooses the right strategy at runtime based on what the user has selected as a route. This keeps the code cleaner, easier to test, and easier to extend later on. This pattern fits our project very well because determining the route is a case of “same goal, different algorithm depending on context.”

2. Composite pattern:

This pattern could be useful because our “places” naturally form some sort of hierarchy: campus → building → floor → room, plus POIs/amenities like washrooms and water fountains that belong to a floor or a building. The Composite design pattern would let the app treat all these elements through one common interface, instead of the situation where we need to have separate logic for campus objects, building objects, floor objects, and room objects. The main benefit we would observe is simplicity – the same features (searching inside a campus, listing children, building a navigation tree in the UI, zooming to a selected place) can work the same way at every level which would avoid some time wasting for the team when implementing their logic. Indeed, without the Composite design pattern, the code tends to become repetitive because each level needs its own special-case handling. Composite could go well with our project, better than many other patterns because the “nested structure” is the core of the domain: you cannot model multi-campus, multi-floor navigation cleanly without thinking in the direction of a tree structure.

3. Adapter pattern:

This pattern could be useful because “where is the user?” can come from different places and positions, and they don’t really behave the same way. Indeed, for outdoors, the GPS tends to work really well. As for indoors, the GPS can be inaccurate, so the app might need alternatives like a manual location selection (“I’m on Floor 8 of Hall building”). With an Adapter, the GPS, indoor positioning, or manual pin can each be “adapters” that plug into the same interface. This could be very relevant to our project because the indoor/outdoor switching could be a real navigation problem on

campus, and adapters reduce vendor/SDK dependency while keeping the navigation logic stable and simple (simple and stable makes it easier to test and read for members of the team).

4. Factory pattern:

This design pattern could be useful because our app will likely handle the POI/amenity categories (water fountains, washrooms, elevators, exits, accessibility ramps, etc.), and each category may need different UI behavior: a different icon, different label text, different details panel, maybe different filtering rules, etc. If these mappings are scattered around the codebase, the app can become very messy and code smells can become more and more prominent, which makes the code harder to maintain. Also, adding a new amenity type could force changes in many files. Indeed, the Factory design pattern centralizes this creation logic in one place. That makes the system cleaner and safer to extend, which could make this design pattern very important in our project. This pattern fits our project better than many others because POI types are practically guaranteed to grow over time, and factories are a simple way to make that growth controlled and organized, helping with the team's time management and sprint delivery.

5. Facade pattern:

This pattern could be very useful because, normally, navigation features require coordination across many parts of the system – searching for a room, selecting a campus/building/floor, converting a user position into the nearest node/point, computing a route, recalculating the route when the user deviates, updating the UI, etc. If every one of the screen/features calls all these components directly, the app ends up with duplicated code and incorrect or inconsistent behavior between screens (for example, one screen updates the floor correctly while another completely forgets to). A Facade would provide a single “Navigation API” that exposes simple operations like `setDestination()`, `getRoute()`, etc. while hiding the internal steps and dependencies. This is useful for our project because it keeps the app consistent and easier to integrate. Indeed, it also makes it simpler and cleaner making for a better code readability for the members of our team.